

UNIVERSITÀ DEGLI STUDI DI ROMA “TOR VERGATA”

Macroarea di Ingegneria

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

INGEGNERIA DEL SOFTWARE E PROGETTAZIONE WEB (ISPW)

CIBOAMICO

*PLATFORM FOR MANAGING FOOD AT HOME, REDUCING WASTE AND
OPTIMISING THE SHOPPING EXPERIENCE*

SUPERVISORS:

PROF. DAVIDE FALESSI

PROF. GUGLIELMO DE ANGELIS

AUTHOR:

MICHELE DAMIANO

STUDENT ID: 0324516

MICHELE.DAMIANO@STUDENTS.UNIROMA2.EU

ACADEMIC YEAR 2025/2026

SUBMISSION DATE: AUGUST 13, 2026

Contents

1	Software Requirements Specification	3
1.1	Introduction	3
1.1.1	Purpose of the document	3
1.1.2	System overview	3
1.1.3	Hardware and software requirements	4
1.1.4	Related systems	4
1.2	User Stories	5
1.2.1	US-1	5
1.2.2	US-2	5
1.2.3	US-3	5
1.3	Functional Requirements	6
1.3.1	Glossary of Domain Terms	6
1.4	Use Cases	7
1.4.1	Use case diagram	7
1.4.2	Internal Steps	7
2	Storyboards	9
2.1	Authentication and Personal Area	9
2.2	Purchases and Marketplace of the Marketplace	11
3	Design	12
3.1	Class Diagram	12
3.1.1	VOPC (Analysis)	12
3.2	Design Patterns	13
3.2.1	Design Pattern Descriptions	14
3.3	Activity Diagram	16
3.4	Sequence Diagram	17
3.5	State Diagram	17
4	Testing	18

5	Exceptions	19
5.1	Exception Hierarchy	19
5.2	Wrapping (Exception Chaining)	19
5.3	Messages	19
6	Persistency	20
6.1	DB Persistence Layer	20
6.2	FS (File System) Persistence Layer	20
6.3	DEMO (In-Memory) Persistence Layer	21
7	Video and Links	22
7.1	GitHub Repository	22
7.2	SonarCloud	22
7.3	Demonstration video	22

Chapter 1

Software Requirements Specification

1.1 Introduction

1.1.1 Purpose of the document

This document defines the software requirements of **CiboAmico**, the final project of the ISPW course (University of Rome “Tor Vergata”, A.Y. 2025/2026, Prof. Davide Falessi, Prof. Guglielmo De Angelis).

The purpose is to state what the system shall do in a clear and verifiable way, leaving room for design and testing. The document serves both developers, who must keep the Boundary-Control-Entity (BCE) architecture, and the evaluators, who compare the functional flows against the initial requests.

1.1.2 System overview

CiboAmico is a desktop JavaFX application that helps manage food at home, avoid waste and optimise the shopping experience by connecting the user with local vendors. The system follows the Boundary-Control-Entity (BCE) pattern and involves four actors: Customer, Vendor, Nutritionist and Administrator. Two external systems are reached through an Adapter: OpenFoodFacts (reading nutritional data from the barcode) and JakartaMail (sending notifications). Persistence is interchangeable at runtime between a *Demo* (in-memory) and a *Full* (MySQL via JDBC) mode.

The system has three functional areas:

- **Pantry:** the Customer keeps track of the food at home, sees the expiry dates and is warned before the food goes bad;
- **Recipes:** the system proposes recipes compatible with the available ingredients, substituting the missing ones;

- **Marketplace:** the Vendors publish products, the Customers place direct orders (one buyer, one vendor) and the Administrator approves vendors and recipes.

1.1.3 Hardware and software requirements

The application is cross-platform thanks to the JVM.

Software requirements:

- Java 17+
- Draw.io (for creating the UML diagrams)
- IDE: IntelliJ IDEA or Eclipse
- MySQL Server

Hardware requirements:

- CPU: Dual-Core 2.0 GHz
- RAM: 3 GB
- Disk Space: 2 GB
- Operating System: Windows, macOS, Linux (compatible with Java 17+)

1.1.4 Related systems

SYSTEM 1: Too Good To Go

Too Good To Go is a marketplace that allows users to buy unsold food at a reduced price from local shops and restaurants, reducing commercial food waste. Orders are collected as a surprise “Magic Box“, whose content is known only after the purchase.

Pros:

- Wide network of partners with geolocated offers in real time.
- Actively reduces commercial waste by offering food at reduced prices.

Cons:

- It does not track the home pantry: after the purchase it does not help manage the expiry dates.
- The purchase is a surprise *Magic Box*: the user does not choose the single products nor plan recipes or diets.

SYSTEM 2: SuperCook

SuperCook is an online recipe engine that finds recipes from the ingredients the user already has at home, with filters by dish type and dietary intolerances.

Pros:

- Powerful matching: it quickly finds many recipes from the inserted ingredients.
- Large database with filters by dish type and intolerances.

Cons:

- It has no integrated purchase channel: the user must buy the missing ingredients by themselves.
- It does not automatically substitute the ingredients, excluding recipes that would be possible with small variations.

CiboAmico aims to combine pantry tracking, recipe generation with substitutions and direct purchase from local producers in a single platform, providing features dedicated to customers, vendors, nutritionists and administrators while keeping a modular and extensible architecture.

1.2 User Stories

1.2.1 US-1

As a Customer, I want to buy products from verified local vendors, so that I can reduce waste and receive fresh food quickly.

1.2.2 US-2

As a Customer, I want to apply a promotional voucher code during the checkout process, so that I can get a discount on my order.

1.2.3 US-3

As a Vendor, I want to publish my products in the marketplace, so that customers can find and order them.

1.3 Functional Requirements

FR-1: Order Management

The system shall provide the registration of a purchase order associating the customer with the vendor.

FR-2: Voucher Management

The system shall provide the application of a promotional voucher to the current order, validating the expiry date and calculating the resulting discounted total.

FR-3: Product Publishing

The system shall provide the publishing of a new product in the marketplace by an approved vendor.

1.3.1 Glossary of Domain Terms

Catalog : set of products published by approved vendors and made available for purchase in the marketplace.

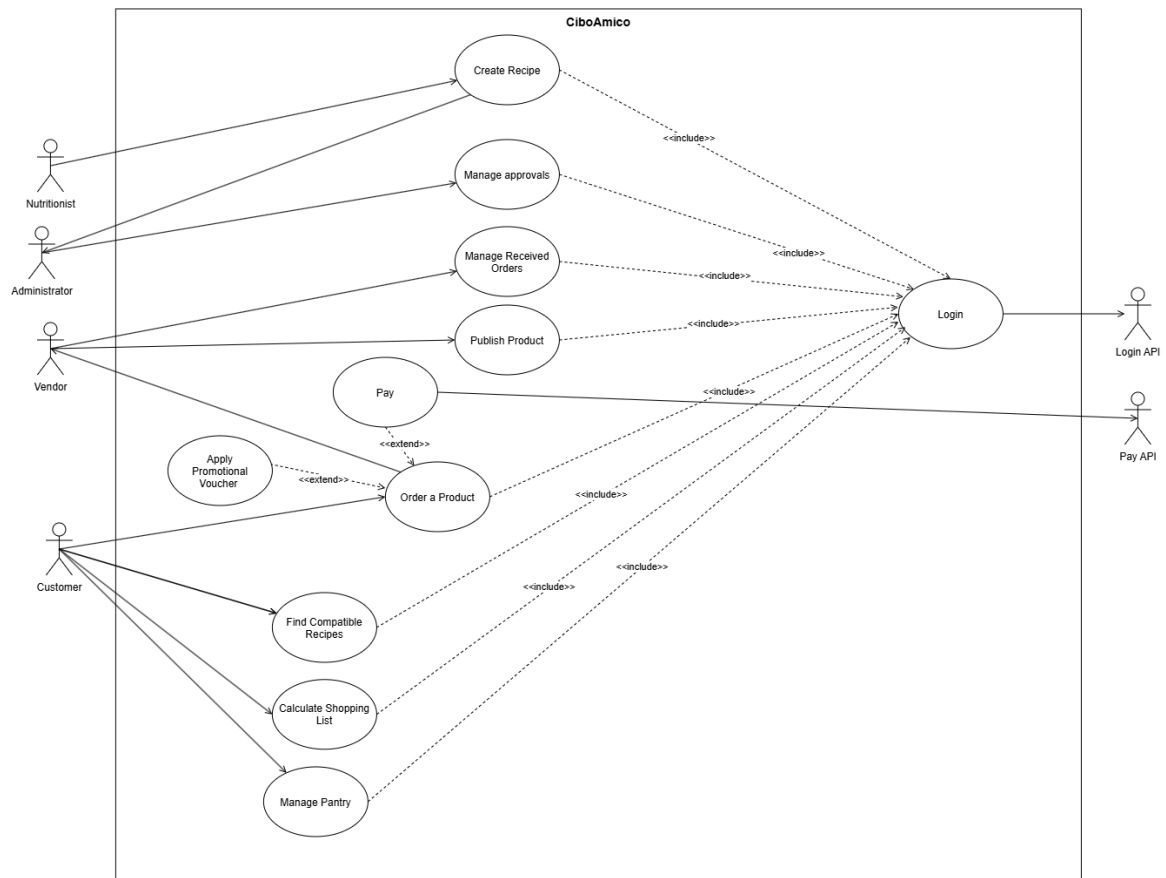
Promotional voucher : discount code associated with a specific vendor, valid within a time window and applicable to an order of that vendor.

Direct order : transaction that associates a single buying customer with a single vendor, without passing through a shared cart or intermediaries.

Product availability : quantity of a product currently sellable, which can never be negative and is updated after every purchase.

1.4 Use Cases

1.4.1 Use case diagram



1.4.2 Internal Steps

Order a Product

Main Flow:

1. The customer accesses the marketplace via Login.
2. The system displays the available products with price and remaining stock.
3. The customer selects a product from the catalog and starts the checkout.
4. The system loads the selected product into the current order and computes the total amount.
5. The customer confirms the purchase.
6. The system requests the authorization for the debit of the computed amount from the Payment Service.
7. The system registers the order in the “CREATED” state, decreases the remaining stock of the product and notifies the vendor.

Preconditions: the customer is authenticated; at least one product is available in the catalog.

Alternative Flow (Extensions):

- **3a. Product No Longer Available:** the selected product is not present in the catalog anymore; the system displays an error message.
- **4a. Promotional Voucher Request:** the customer requests to apply a promotional voucher to the current order; the system validates the temporal validity of the voucher and its association with the vendor of the selected product, recalculates the total amount applying the discount and resumes from step 5.
- **6a. Payment Rejected:** the Payment Service returns a negative outcome; the system cancels the current order and displays a failure message, returning the flow to step 4.

Chapter 2

Storyboards

2.1 Authentication and Personal Area

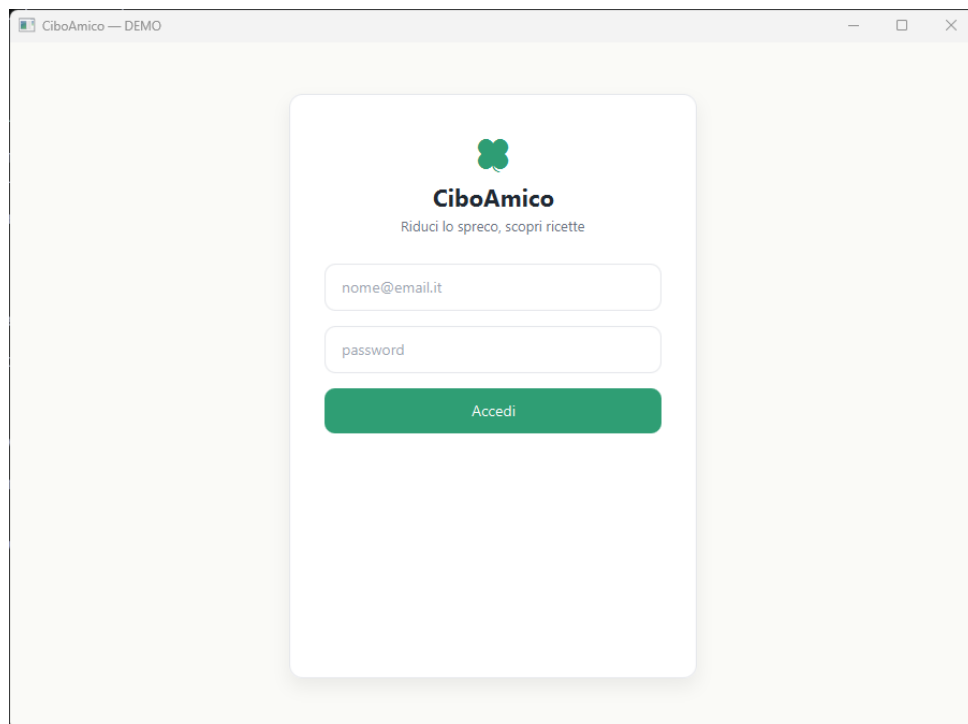


Figure 2.1: Authentication screen of the system.

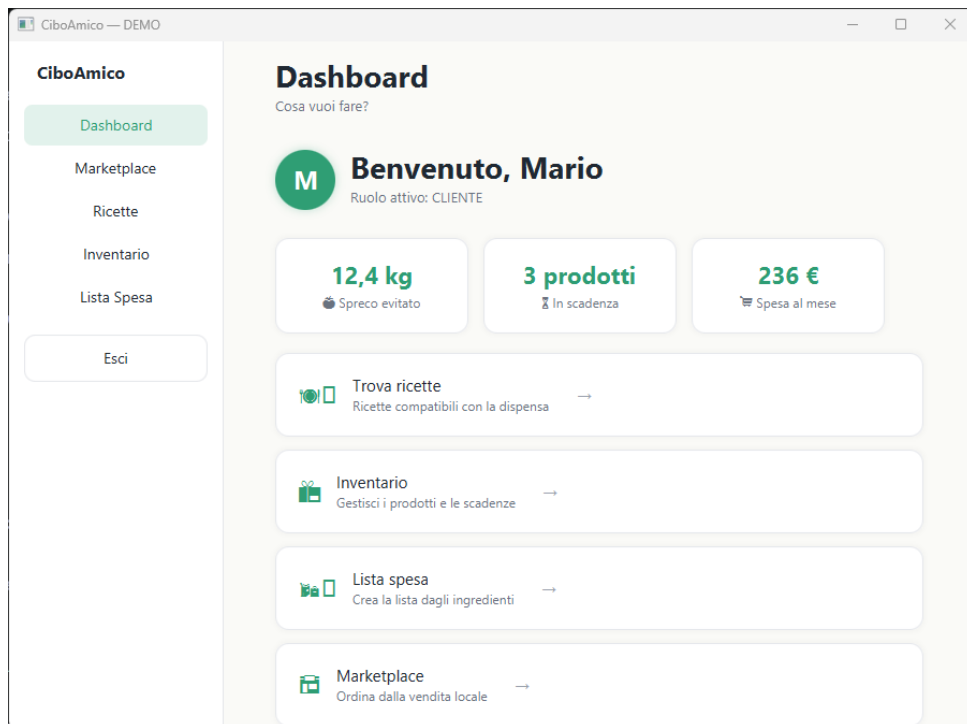


Figure 2.2: Main dashboard (customer).

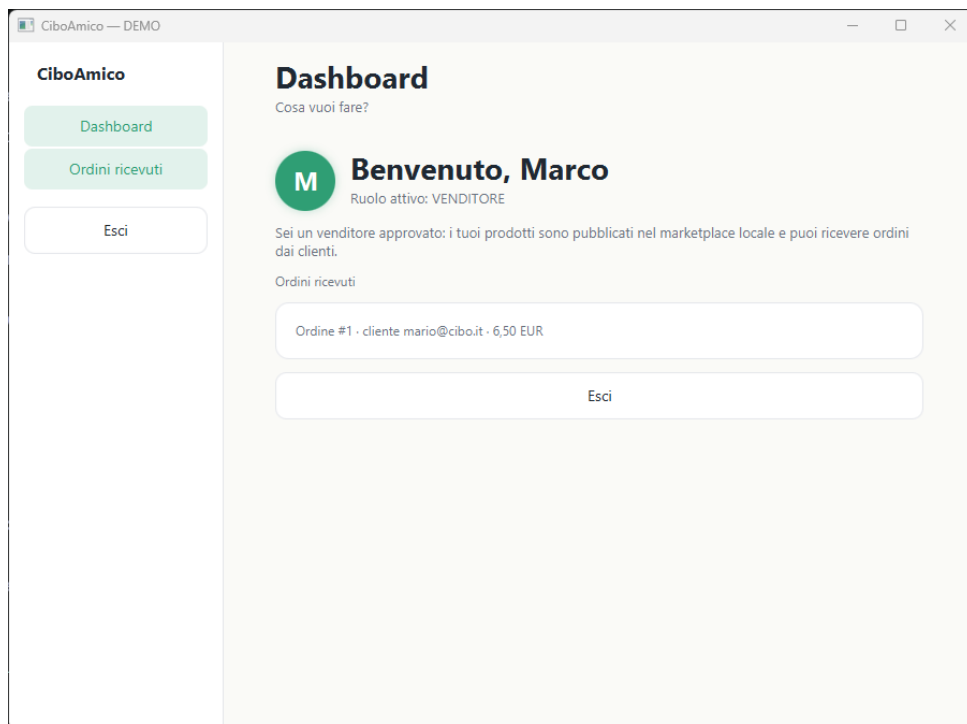


Figure 2.3: Vendor dashboard: received orders list.

2.2 Purchases and Marketplace of the Marketplace

The screenshot shows the 'Marketplace locale' interface in the CiboAmico application. The left sidebar contains navigation links: Dashboard, Marketplace (highlighted), Ricette, Inventario, Lista Spesa, and an Esci button. The main content area is titled 'Marketplace locale' with a subtitle 'UC-04 · ordina dalla vendita locale'. A green button 'Aggiorna catalogo' is at the top. Below it, a message states '2 prodotti disponibili nel marketplace locale.' followed by the heading 'Catalogo prodotti'. Two product cards are displayed: 'Miele locale' (6,50 EUR - 20 disponibili) and 'Pomodori' (2,20 EUR - 50 disponibili). To the right is a purchase panel with a 'Prodotto da ordinare' dropdown menu (showing 'Scegli un prodotto'), a 'Buono promozionale (opzionale)' section with a 'Codice buono (opzionale)' input field, and two green buttons: 'Applica buono' and 'Ordina prodotto'.

Figure 2.4: Marketplace: local product catalog and purchase panel.

The screenshot shows the 'Pagamento' (Payment) interface in the CiboAmico application. The left sidebar is identical to the previous screen. The main content area is titled 'Pagamento' with a subtitle 'UC-04 · autorizzazione all'addebito'. A summary box at the top shows 'Riepilogo: Miele locale — 6,50 EUR'. Below this are input fields for 'Numero carta', 'Intestatario', 'Scadenza (MM/AA)', and 'CVV'. At the bottom are two large green buttons: 'Paga' and 'Annulla'.

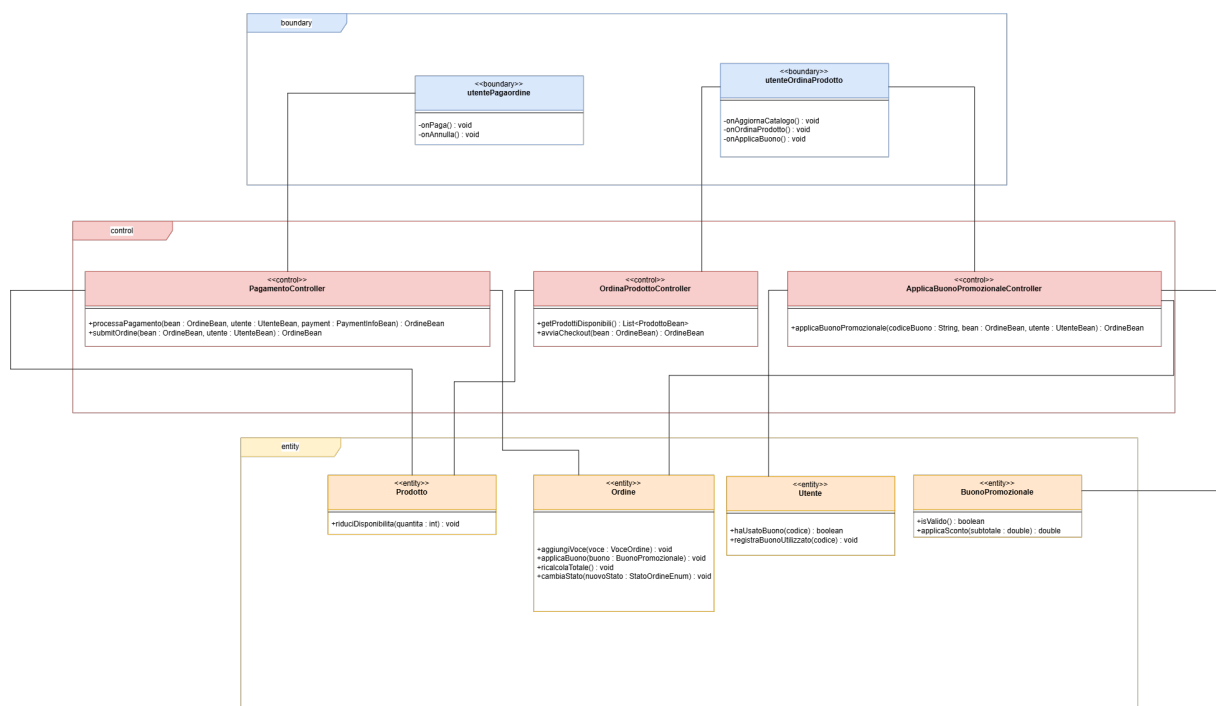
Figure 2.5: Payment screen (card data and debit authorization).

Chapter 3

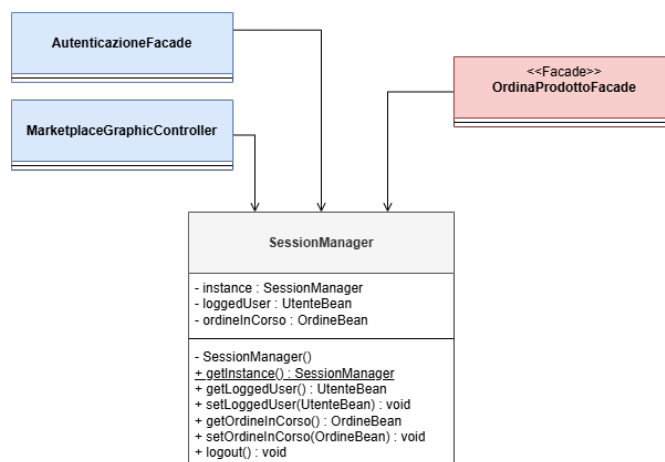
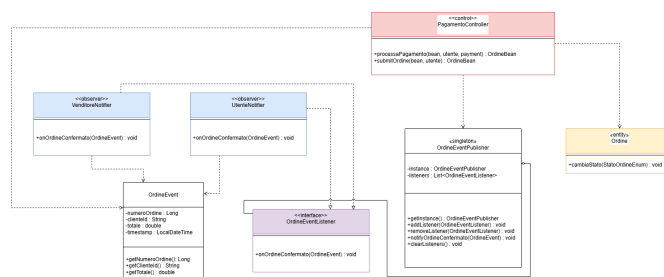
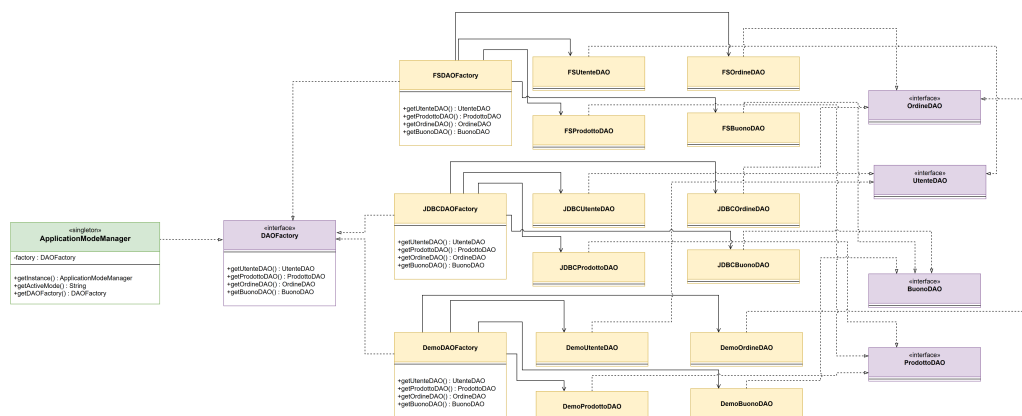
Design

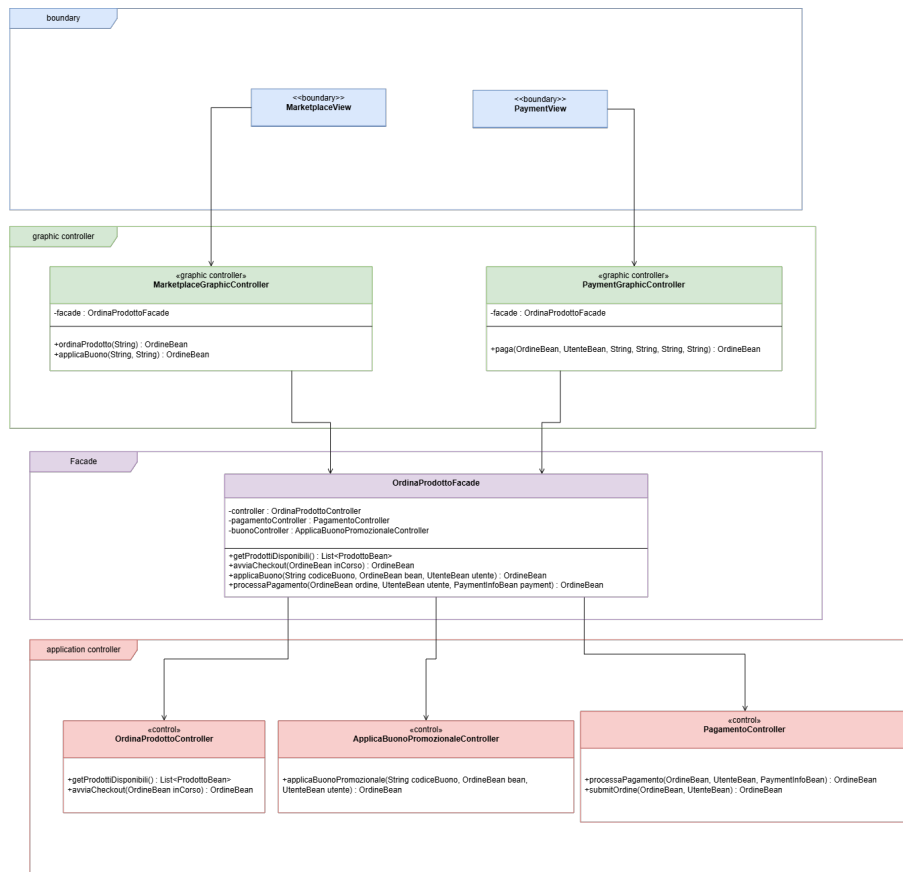
3.1 Class Diagram

3.1.1 VOPC (Analysis)



3.2 Design Patterns





3.2.1 Design Pattern Descriptions

Abstract Factory Pattern – Persistence Layer

The persistence layer is managed via the Abstract Factory pattern to ensure execution-environment flexibility. The `DAOFactory` interface defines the contract for creating Data Access Objects, such as `getOrdineDAO()` and `getProdottoDAO()`. Based on the `persistence_type` configuration, the concrete factories `JDBCDAOFactory`, `FSDAOFactory` or `DemoDAOFactory` instantiate the appropriate storage implementation. This enables the system to switch between database, file-system or in-memory persistence without changing the client code.

Observer Pattern – Notification System

I implemented the Observer pattern to decouple the order processing from the notification logic. The `OrdineEventPublisher` class serves as a Singleton Subject, managing a `List` of `OrdineEventListener` observers. Upon successful order submission, the publisher triggers the `onOrdineConfermato()` method across all registered listeners. This architecture ensures a loose coupling between the core logic and its side effects, strictly adhering to the Open/Closed Principle.

Singleton Pattern – Sessions and Mode

`SessionManager` (user session) and `ApplicationModeManager` (application mode) adopt a classic Singleton with lazy initialization and synchronization on `getInstance()`, in line with the reference projects: no perceivable cost in a desktop client, no SonarCloud warning suppression.

Facade Pattern – Order Flow

`OrdinaProdottoFacade` exposes to the graphic controller a single interface for the order flow (`getProdottiDisponibili`, `avviaCheckout`, `applicaBuono`, `processaPagamento`) and hides the business subsystem, formed by the base controller and by the extension controllers. The client does not know the order in which the methods of the application controllers are called; the operations of the extensions live only in their respective sub-controllers and the Facade invokes them directly (no useless pass-through).

3.3 Activity Diagram

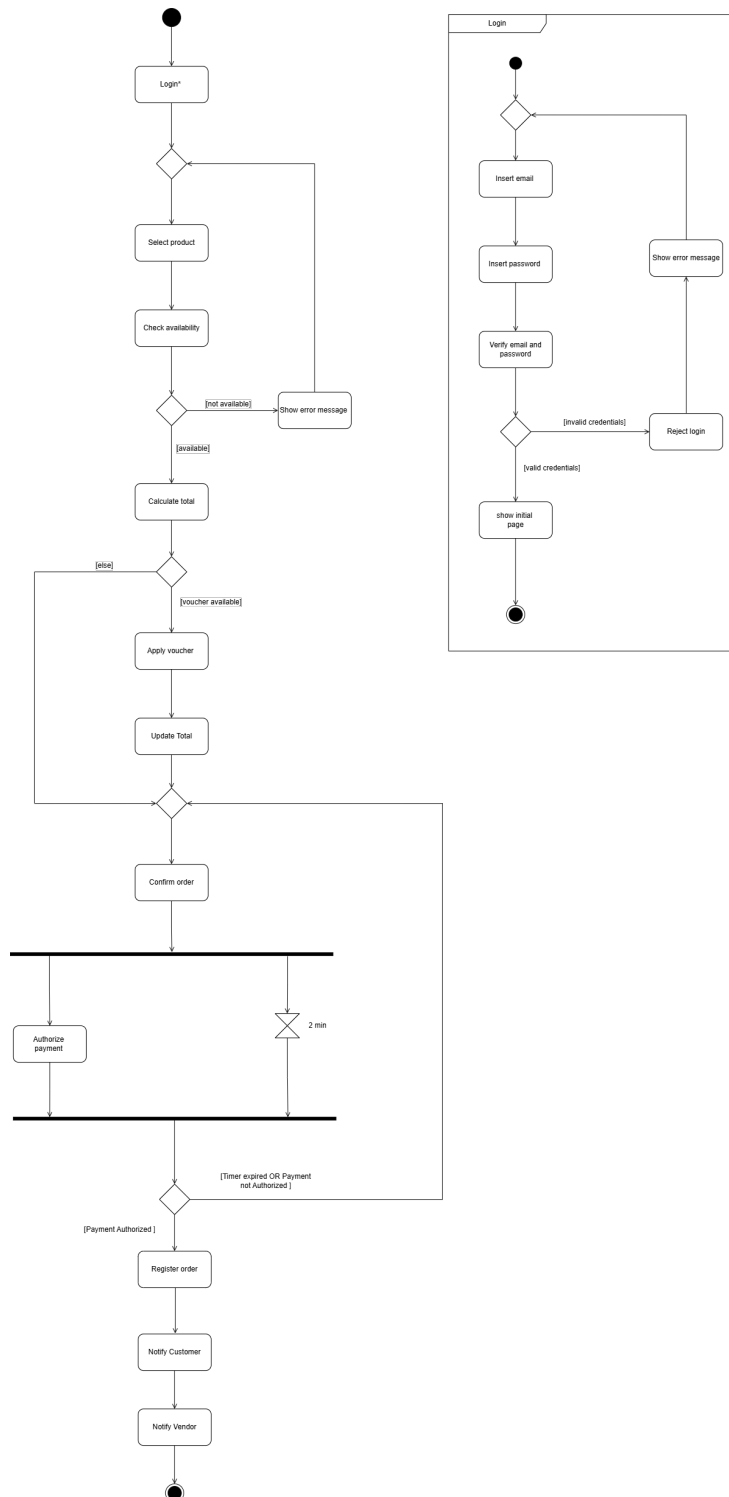


Figure 3.1: Activity diagram of the order flow: availability check, voucher application, payment authorization and sequential notification to buyer and vendor.

3.4 Sequence Diagram

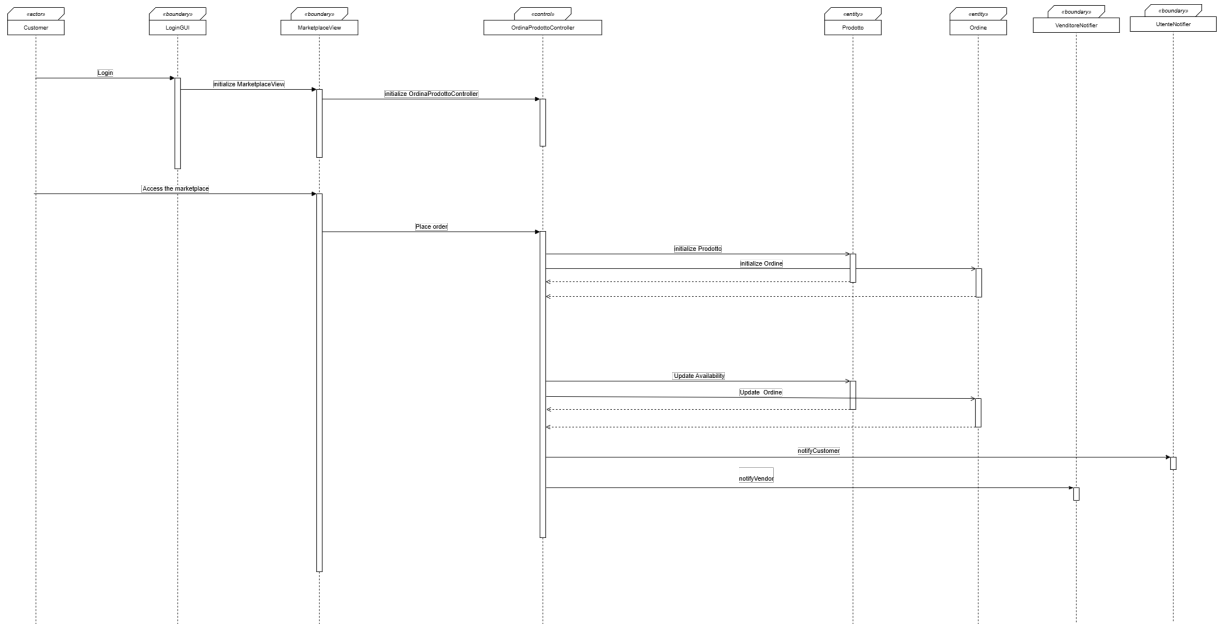


Figure 3.2: Sequence diagram of the order flow.

3.5 State Diagram

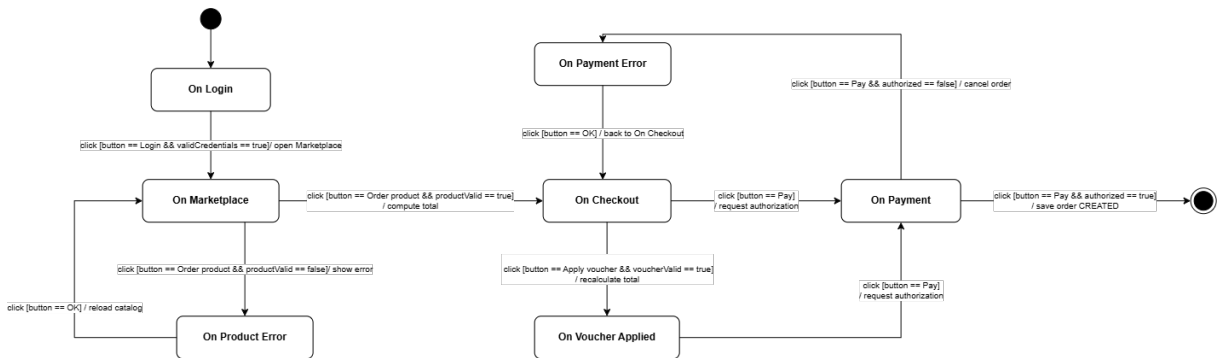


Figure 3.3: State diagram of the order flow: navigation and alternative flows.

Chapter 4

Testing

Voucher & Order Safety I verified that the system handles edge cases gracefully. The `ApplicaBuonoPromozionaleController` correctly identifies invalid, empty or null voucher codes without crashing, and applying a voucher only to an order of the matching vendor is now exception-safe, ensuring overall stability.

Order Lifecycle Tests on `OrdinaProdottoController` confirm that a new order is generated correctly with valid product and vendor data. I also ensured that the availability check rejects an out-of-stock product and that the stock is reduced at purchase time without negative values.

Payment & Authorization I verified both the authorized and the rejected branch of the payment. The system correctly blocks invalid card data (CVV, non-positive amount, missing order) by throwing `BusinessValidationException`, leaving the order unchanged when the payment is rejected.

Authentication The authentication system now strictly enforces email formatting. It supports valid addresses, correctly rejects malformed inputs (like missing @ or empty strings) by throwing `AutenticazioneException`, and the `logout` correctly clears the session.

Observer & Persistence I confirmed that the `OrdineEventPublisher` operates as a true Singleton subject, notifying the registered observers only through the read-only DTO. Additionally, the Demo and File System DAO layers keep user, product and order data consistent throughout the create-read-update cycle.

Chapter 5

Exceptions

5.1 Exception Hierarchy

The system implements a custom exception hierarchy based on a common abstract class, `CiboAmicoException`, which extends `java.lang.Exception` (checked). Its subclasses model the domain errors: `BusinessValidationException` for violated business rules, with `AutenticazioneException` and `InvalidStateTransitionException` as specialized cases, and `DAOException` for persistence failures. The payment gateway defines a separate `PaymentRejectedException` (extends `Exception`) for an unauthorized transaction.

5.2 Wrapping (Exception Chaining)

The `(String message, Throwable cause)` constructor of `CiboAmicoException` and its subclasses allows you to:

- Add semantic context to the error.
- Preserve the original stack trace.
- Maintain the causal chain for debugging purposes.

The DAOs use it to wrap the low-level error (`SQLException`, `IOException`) in a `DAOException`, so that `getCause()` preserves the original cause, which never propagates to the boundaries.

5.3 Messages

Texts are centralized in `ExceptionMessagesEnum` (logs) and `UserErrorMessagesEnum` (user), without duplicated strings at the throw points.

Chapter 6

Persistence

6.1 DB Persistence Layer

The database persistence layer utilizes MySQL as the relational DBMS. Data management is handled through specific DAO (Data Access Object) classes (e.g., `JDBCUtenteDAO`, `JDBCOrdineDAO`), which execute SQL queries to perform CRUD (Create, Read, Update, Delete) operations.

Primary Entities:

- **Users:** Registered users categorized by roles (Customer, Vendor).
- **Orders:** Transactional records including state and total amount.
- **Products:** Marketplace items in the catalog.
- **Vouchers:** Discount codes (supporting both percentage-based and fixed-amount deductions).

Configuration: The system is configured via the `config.properties` file by setting `persistence_type = JDBC` and providing the necessary MySQL credentials. Transactions are atomic via `ConnectionManager` (`setAutoCommit(false)` with `commit/rollback`).

6.2 FS (File System) Persistence Layer

The FS persistence layer stores the data within JSON files located in the `data/` folder. Each entity is mapped to a dedicated file featuring records (e.g., `utenti.json`, `ordini.json`, `prodotti.json`).

Management: The `GsonConfig` class provides a consistent JSON serializer (helper for `LocalDate/LocalDateTime`) used by all the FS DAOs to guarantee the correct reading/writing of the records.

Configuration: This mode is activated by setting `persistence_type = FS` in the configuration file.

6.3 DEMO (In-Memory) Persistence Layer

The DEMO persistence layer maintains data exclusively within the volatile memory (RAM), without any permanent storage. Data structures are populated with predefined default values upon application startup and are cleared when the execution terminates.

Use Cases: This layer is intended for unit testing, rapid prototyping, and software demonstrations that do not require database configuration.

Configuration: This mode is activated by setting `persistence_type = DEMO` (default).

Chapter 7

Video and Links

7.1 GitHub Repository

<https://github.com/Damiano-o/ciboamico>

7.2 SonarCloud

https://sonarcloud.io/dashboard?id=Damiano-o_ciboamico2

7.3 Demonstration video

The 2-minute video shows the main functionalities (login, marketplace order with voucher, payment, vendor notification). It is available as `ciboamico-demo.mpeg` in the project root and on YouTube at: <https://youtu.be/AuZxufdVJ7E>